

Architecture.md v1.1

Chapter 7 — Database & Persistence Architecture

This chapter defines how AI OS stores, retrieves, protects, and manages persistent data.

The Persistence Layer is responsible only for storage. It does not perform reasoning, routing, or business logic.

7.1 Purpose

The Persistence Layer provides a centralized, reliable, and secure storage system for AI OS.

Its objectives are to:

- Store all persistent user data.
 - Preserve long-term memory.
 - Maintain project information.
 - Support future semantic search.
 - Ensure data consistency.
 - Protect sensitive information.
-

7.2 Design Philosophy

Persistence follows four principles:

Centralized

All persistent data is managed through the Memory Service.

Abstracted

No module communicates directly with a database engine.

Every read and write operation passes through the Memory Service.

Replaceable

The storage engine may change without affecting higher layers.

Examples:

SQLite

↓

PostgreSQL

↓

Cloud Database

↓

Distributed Database

The architecture remains unchanged.

Secure

Sensitive information is encrypted and access-controlled.

7.3 Database Architecture

```
graph TD
    User --> Router
    Router --> Jarvis_Friday["Jarvis / Friday"]
    Jarvis_Friday --> Agent_Manager["Agent Manager"]
    Agent_Manager --> Memory_Service["Memory Service"]
    Memory_Service --> Persistence_Manager["Persistence Manager"]
    Persistence_Manager --> Database_Engine["Database Engine"]
```

Worker Agents never access the database.

7.4 Storage Technologies

Version 0.1

Primary Database:

SQLite

Reason:

- Lightweight
 - Simple deployment
 - Local-first
 - Zero server configuration
-

Future Versions

Relational Database

PostgreSQL

For:

- Multiple projects
 - Advanced queries
 - Higher performance
-

Vector Database

ChromaDB

Purpose:

- Semantic search
 - Memory retrieval
 - AI embeddings
-

File Storage

Stores:

- Images
- PDFs
- Documents
- Audio
- Videos

The database stores metadata only.

7.5 Persistence Manager

The Persistence Manager is an internal component of the Memory Service.

Responsibilities:

- Read data
- Write data
- Update records
- Delete records
- Execute transactions
- Handle database migrations

No Worker Agent interacts with it directly.

7.6 Data Domains

Persistent information is organized into logical domains.

User

- Profile
 - Preferences
 - Devices
 - Settings
-

Memory

- Long-term memories
 - Session summaries
 - Knowledge
-

Projects

- Project metadata
 - Status
 - Documents
 - Decisions
 - Milestones
-

Tasks

- Pending
 - Active
 - Completed
 - Archived
-

Analytics

- Reports
 - Performance history
 - Metrics
-

Business

- Clients
 - Revenue
 - Expenses
 - Business notes
-

Trading

- Watchlists
- Journals
- Positions
- Strategies

(No live trading credentials are stored without explicit configuration.)

Logs

- Requests
- Errors
- Events

- Performance metrics
-

7.7 Transactions

Every write operation must occur inside a transaction.

Transaction properties:

- Atomic
- Consistent
- Isolated
- Durable

If any step fails:

Entire transaction is rolled back.

Partial writes are prohibited.

7.8 Concurrency

Version 0.1

Execution Model:

Sequential

Race conditions are minimized through serialized writes.

Future Versions

The Persistence Layer shall support:

- Concurrent reads
 - Transaction locking
 - Optimistic concurrency control
 - Background synchronization
-

7.9 Data Integrity

Every record shall include:

- Unique Identifier (UUID)
- Creation Timestamp
- Last Updated Timestamp
- Version Number
- Owner (AI OS)
- Category

No anonymous records are permitted.

7.10 Soft Delete Policy

Deletion follows two stages.

Stage 1

Archive

The record is marked inactive.

Stage 2

Permanent Delete

Requires:

- Explicit user approval
- Confirmation
- Logging

Critical memories cannot be permanently removed accidentally.

7.11 Backup Strategy

Future versions shall support:

- Automatic backups
- Scheduled backups

- Manual restore
- Snapshot creation

Backups are independent of the active database.

7.12 Data Encryption

Sensitive information shall be encrypted at rest.

Examples:

- API Keys
- Tokens
- Credentials
- Financial information

Plain-text storage of secrets is prohibited.

7.13 Persistence Rules

Allowed:

Memory Service

↓

Database

Persistence Manager

↓

Database

Forbidden:

Worker Agent

↓

Database

Jarvis

↓

Database

Friday

↓

Database

Router

↓

Database

These violations are architecture-breaking.

7.14 Database Migrations

Every schema change requires:

- Version identifier
- Migration script
- Rollback support

Direct manual schema modifications are prohibited.

7.15 Database Health

The Persistence Layer shall monitor:

- Connection status
- Read latency
- Write latency
- Storage usage
- Transaction failures

Health reports are sent to the Logging Service.

7.16 Future Expansion

The Persistence Layer is designed to support:

- Cloud synchronization
- Multiple storage providers
- Distributed databases
- Semantic memory indexing
- Offline-first synchronization
- Multi-device support
- Incremental backups

These features should be introduced without changing higher architectural layers.

7.17 Design Rules

The Persistence Layer must always satisfy the following:

- Single point of persistence.
 - Database abstraction.
 - Transaction safety.
 - Data consistency.
 - Secure storage.
 - Version-controlled schema.
 - Replaceable storage engines.
-

End of Chapter 7

Next Chapter: Chapter 8 — Event System & Workflow Engine